

T1 - Análise de Complexidade de Algoritmos de Ordenação

Ariel Alves da Silva
Bach. Ciências da Computação - ICMC (USP)
8847378
arielalves@usp.br

I. INTRODUÇÃO

O atual relatório consiste na análise de dois algoritmos de ordenação, sendo um deles de complexidade $O(n^2)$ e o outro de complexidade $O(n \log n)$ ou superior. Será realizada análise teórica (análise assintótica) e análise empírica, utilizando medições de tempo de execução para 4 diferentes casos de organização e vetores: aleatórios, aleatórios com repetição, ordenados e ordenados de modo decrescente.

Além disso, será realizada uma análise dos resultados de uma tabela de Baseline de algoritmos proposta pelo professor, de modo a avaliar algoritmos desconhecidos e suas peculiaridades.

Esse relatório se divide em 4 partes: definição dos algoritmos utilizados, análise teórica, resultados experimentais, análise experimental, resultados experimentais do baseline e análise do baseline.

II. ALGORITMOS UTILIZADOS

A. Insertion Sort

O insertion sort consiste em ordenar um subarray que começa com tamanho $n=1$ e se inicia no primeiro índice, inserindo o elemento imediatamente à direita do subarray dentro do subarray (ordenado) a cada iteração.

insertionSort():

```
1 para i de 1 até n faça:
2     aux ← A[i]
3     j ← i - 1
4     enquanto (j ≥ 0) ∧ (aux < A[j]) faça:
5         A[j + 1] ← A[j]
6         j ← j - 1
7     fim_enquanto
8     A[j + 1] ← aux
9 fim_para
```

B. Heap Sort na modalidade max_heap

Um vetor $A[1..n]$ é um *max_heap* se $A[j] \geq A[2j]$ e $A[j] \geq A[2j + 1]$ sempre que os índices não ultrapassem n [1].

O heap sort consiste em organizar os elementos do vetor de modo a construir uma heap. Ao construir a heap, $V[0]$ será o maior elemento do subarray, e será colocado fora da heap (imediatamente após). Fazendo-se isso consecutivamente, o array terminará ordenado.

heapSort():

```
1 para i de  $\lfloor \frac{n}{2} \rfloor - 1$  até 0 faça:
2     fazHeap(A, n, i)
3 fim_para
4 para i de n - 1 até 0 faça:
5     troca(A[0], A[i])
6     fazHeap(A, i, 0)
7 fim_para
```

O algoritmo fazHeap é simples: se $A[i]$ é maior ou igual que seus filhos então não é preciso fazer nada; senão, troque $A[i]$ com o maior dos filhos e repita o processo para o filho envolvido na troca [1].

fazHeap(A, n, i):

```
1 L ← 2 · i + 1
2 R ← 2 · i + 2
3 C ← i
4 se (L < n) ∧ (A[L] < A[C]): C ← L
5 se (R < n) ∧ (A[R] < A[C]): C ← R
6 se (C ≠ i):
7     troca(A[C], A[i])
8     fazHeap(A, n, C)
```

III. ANÁLISE TEÓRICA

A. Insertion Sort

No insertion sort, a única condição capaz de interromper o loop interno é a condição $A[j] \leq \text{aux}$. Com base nela serão definidos o pior, melhor e caso médio.

a) Pior caso:

No pior caso, a condição $A[j] \leq \text{aux}$ só é verdadeira na última comparação do loop interno. Assim, o subarray será percorrido por inteiro em todos os loops. A quantidade de comparações total é:

$$\sum_{k=1}^{n-1} k = \frac{(n-1)[(n-1)-1]}{2} = \frac{n^2 - 3n + 2}{2} \quad (1)$$

E, portanto, a complexidade é $\mathcal{O}(n^2)$.

b) *Melhor caso:*

No melhor caso, a condição $A[j] \leq \text{aux}$ sempre é verdadeira na primeira comparação, interrompendo o loop interno o mais cedo possível. Esse caso é sempre válido se, e somente se o array está ordenado.

Como cada loop interno será executado somente uma vez com complexidade $\mathcal{O}(1)$ (apenas uma comparação simples) e o loop externo é executado de 1 até n , a complexidade é linear: $\Omega(n)$.

c) *Caso médio:*

Para definir o caso médio, assume-se que para um elemento na posição i , em média, metade dos elementos anteriores são maiores do que ele. Portanto, ele precisará recuar $\frac{i}{2}$ posições para encontrar seu lugar correto. Assim, cada loop interno será executado pela metade.

Para a metade de uma iteração ímpar, será considerado o teto da divisão:

$$(\text{iteração, operações}) = \{(1, 1), (2, 1), (3, 2), (4, 2), \dots\} \quad (2)$$

Assim, por indução infinita, podem-se assumir duas fórmulas para o total de operações:

$$\sum_{k=1}^{n-1} \left\lceil \frac{k}{2} \right\rceil = \begin{cases} 2 \cdot \frac{n \cdot \lceil \frac{n}{2} \rceil - 1}{2}, & n=\text{par} \\ 2 \cdot \frac{n-1 \cdot \lceil \frac{n-1}{2} \rceil - 1}{2} + \lceil \frac{n}{2} \rceil, & n=\text{ímpar} \end{cases} \quad (3)$$

Onde o termo de maior grau das duas fórmulas é de segundo grau e, portanto, a complexidade no caso médio é $\mathcal{O}(n^2)$.

B. Heap Sort

No heap sort, a única condição capaz de reduzir a quantidade de operações é a de $C = i$. Para cada chamada principal de `fazHeap()`, o máximo de iterações possíveis (por recursão) é $\lceil \log_2(n) \rceil$, com n sendo o número de elementos.

a) *Pior caso:*

Para o pior caso, será considerado que sempre haverá uma troca ($C \neq i$ sempre é válido) para toda chamada da função `fazHeap()`.

No primeiro loop, i vai de $\lceil \frac{n}{2} \rceil - 1$ até 0. Para cada iteração, a “árvore parcial” terá $n - i$ elementos. De maneira formal:

$$\sum_{k=n-(\lceil \frac{n}{2} \rceil - 1)}^{n-1} \lceil \log_2(k) \rceil \quad (4)$$

Pode-se simplificar o limite superior da somatória para $\frac{n}{2}$ com n par, e retirar a função teto para possibilitar o uso da propriedade de soma de logaritmos.

$$\begin{aligned} \sum_{k=n}^{\frac{n}{2}} \log_2(k) &= \log_2\left(n \cdot (n-1) \cdot \dots \cdot \left(\frac{n}{2}\right)\right) = \\ &= \log_2\left(\frac{n!}{(\frac{n}{2}-1)!}\right) \end{aligned} \quad (5)$$

Aqui, é possível aplicar a aproximação de Stirling [2] para o fatorial, onde $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$. Desenvolvendo a expressão e descartando coeficientes e termos lineares, tem-se a complexidade do primeiro loop no pior caso: $\mathcal{O}(n \log_2 n)$.

No segundo loop, a lógica segue sendo a mesma, porém com i de $n-1$ até 0. Isto muda somente os coeficientes e termos da expressão, de modo que a função principal continua sendo $\mathcal{O}(n \log_2 n)$.

Portanto, a complexidade do algoritmo no pior caso é $\mathcal{O}(n \log_2 n)$.

b) *Melhor caso:*

No melhor caso, o array já se encontra na configuração de `max_heap`.

Esta configuração faz com que o primeiro loop nunca execute sub-chamadas dentro da função `fazHeap` ($C = i$ sempre). A complexidade do primeiro loop é $\Omega(n)$.

Já no segundo loop, não é possível afirmar o mesmo. Ao tirar a raiz da árvore e colocá-la imediatamente após a heap, a heap é bagunçada e precisa ser reorganizada. Essa reorganização sempre possui um custo proporcional a $n \log_2 n$, pois sempre haverá chamadas recursivas (mesmo que em menor quantidade no melhor caso) para a ordenação do array. Assim, não existe array onde o segundo loop possui complexidade linear: Da mesma forma que no estudo no pior caso, sua complexidade é $\Omega(n \log_2 n)$.

Portanto, a complexidade algorítmica no melhor caso é $\Omega(n \log_2 n)$.

c) *Caso médio:*

Como a assíntota superior é $\mathcal{O}(n \log_2 n)$ e a assíntota inferior é $\Omega(n \log_2 n)$, conclui-se que no caso médio a complexidade do algoritmo é $\Theta(n \log_2 n)$.

IV. DADOS EMPÍRICOS

Para análise empírica, os algoritmos Insertion Sort e Heap Sort foram escritos em C. Foi utilizada a biblioteca `<time>` para contagem do tempo de execução.

Cada configuração: algoritmo vs tipo de array foi executada 10 vezes e o resultado presente na tabela é a média das dez, de modo a evitar medições incomuns (casos excepcionais captados aleatoriamente que afetam drasticamente o desempenho do algoritmo).

A. Tabelas de tempo relativo

Para facilitar a interpretação das tabelas, o algoritmo mais rápido obtém tempo de execução igual a 1 e o mais lento recebe o valor proporcional (ex: se o mais rápido rodou em 10ms e o mais lento em 20ms, o primeiro recebe 1 e o segundo 2), truncados para uma casa decimal. Os dados completos se encontram no apêndice A (Section IX).

TABLE I
TEMPO DE EXECUÇÃO RELATIVO - ARRAY ORDENADO

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Insertion Sort	1	1	1	1	1
Heap Sort	18.7	29.8	32.43	38.9	40.5

TABLE II
TEMPO DE EXECUÇÃO RELATIVO - ARRAY DECRESCENTE

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Insertion Sort	2.7	8.2	14.2	58.9	106
Heap Sort	1	1	1	1	1

TABLE III
TEMPO DE EXECUÇÃO RELATIVO - ARRAY ALEATÓRIO

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Insertion Sort	1.4	3.7	6.3	24.4	44.8
Heap Sort	1	1	1	1	1

TABLE IV
TEMPO DE EXECUÇÃO RELATIVO - ARRAY ALEATÓRIO COM REPETIÇÃO

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Insertion Sort	1.3	3.4	6.4	24.2	46.9
Heap Sort	1	1	1	1	1

B. Gráficos de tamanho vs tempo

Os gráficos abaixo são referentes aos mesmos valores das tabelas Table IX, Table X, Table XI, Table XII no apêndice A.

Também estão presentes as assintotas de melhor caso e pior caso para comparação visual. As constantes multiplicativas foram definidas arbitrariamente, por tentativa e erro, até se aproximarem dos limites superiores e inferiores.

a) Insertion Sort:

Para o insertion sort, o limite superior é $\mathcal{O}(n^2)$ e o inferior é $\Omega(n)$. Foram definidas as seguintes constantes para os limites assintóticos:

- Limite superior: $(2.2 \cdot 10^{-6})n^2$
- Limite inferior: $(1 \cdot 10^{-4})n$

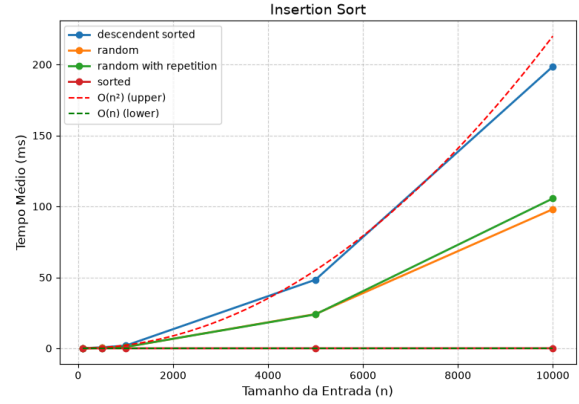


Fig. 1. Tempo de execução para o Insertion Sort com limites assintóticos

b) Heap Sort:

Para o heap sort, tanto o limite superior quanto inferior são $\Theta(n \log_2 n)$. Os limites assintóticos e suas constantes multiplicativas foram definidos como:

- Limite superior: $(2 \cdot 10^{-5}) \cdot n \log_2 n$
- Limite inferior: $(1 \cdot 10^{-5}) \cdot n \log_2 n$

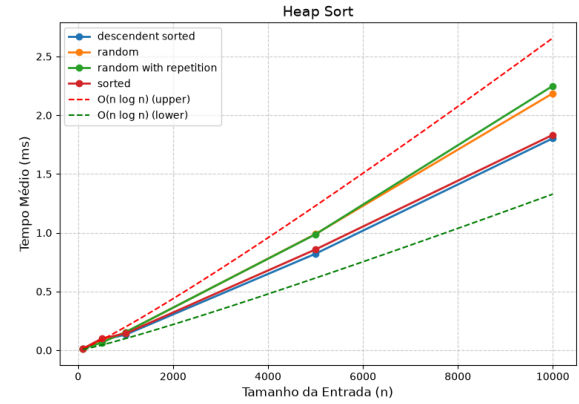


Fig. 2. Tempo de execução para o Heap Sort com limites assintóticos

V. ANÁLISE EMPÍRICA

A. Tabelas de tempo relativo

As tabelas de tempo relativo apresentaram consistência entre as diferentes categorias, de modo a apontar as vantagens e desvantagens de cada algoritmo.

Para arrays ordenados (Table I), o insertion sort obteve um desempenho superior. Isso já era esperado, uma vez que, na análise teórica dos algoritmos, o array ordenado resulta no melhor caso, uma complexidade linear. O heap sort, mesmo com complexidade não-quadrática, ainda assim sofreu com arrays ordenados, o que também era esperado: montar uma heap “bagunça” o vetor já consolidado para ordenar novamente.

Isso mostra a fraqueza do heap sort: apesar de ter limites assintóticos bem restritos, não há vantagem no caso trivial.

Uma solução simples seria verificar se o array já está ordenado, operação que tem complexidade $O(n)$.

Para arrays decrescentes (Table II), o heap sort foi superior em todos os casos. O insertion sort se mostrou muito distante do desempenho do heap sort para $n \geq 1000$. Para $n = 100, 500$, a diferença ainda assim é significativa, mas pequena comparada a quando n começa a ficar muito grande.

Para arrays aleatórios (Table III) e arrays aleatórios com repetição (Table IV), o resultado foi similar. Nota-se um contraste interessante: para $n = 100$, o insertion sort é praticamente equivalente ao heap sort, com pouca diferença. Essa diferença cresce gradualmente e se mantém “baixa” até $n = 1000$. A partir de $n = 1000$ o tempo de execução aumenta drasticamente, e o insertion sort se torna uma péssima opção de ordenação em comparação ao heap sort.

É difícil afirmar sobre qual dos dois tipos de array o insertion sort leva vantagem. Nos dois primeiros tamanhos, o insertion sort possui melhor desempenho no array aleatório com repetição. Já nos dois tamanhos seguintes, o desempenho no array aleatório comum é o melhor. Além disso, os desempenhos estão próximos. Pode-se afirmar que não há diferença significativa, para o insertion sort, no fato de haver repetição ou não no array.

As tabelas contrastam e afirmam as curvas assintóticas definidas anteriormente. O insertion sort tem melhor desempenho em arrays ordenados. Para arrays aleatórios, sejam com repetição ou sem, o insertion sort só consegue competir com o heap sort em valores $n \leq 500$. Para array decrescente ocorre o mesmo, mas o limite de tamanho tolerável é menor. Também é notável o fato do heap sort ter dificuldade em lidar com arrays ordenados.

B. Gráficos de tamanho vs tempo

C. Insertion Sort

No gráfico do insertion sort, há um contraste interessante. O melhor e pior casos propostos teoricamente são evidenciados aqui por uma grande diferença de comportamento.

Para arrays ordenados, o limite inferior é uma função linear com coeficiente baixíssimo, o que denota uma grande eficácia do insertion sort nesse tipo de vetor.

Para arrays decrescentes, o limite superior já tem uma disparidade muito grande, principalmente pelo fato de ser quadrático. É possível ver que o limite superior é adequado para o algoritmo: o pior caso (array decrescente) começa próximo do limite superior mas se afasta conforme n cresce. Isso mostra graficamente que o limite superior quadrático é, de fato, uma assíntota para a função e é válido para $n \rightarrow \infty$ (por análise gráfica).

Para arrays aleatórios, é possível ver uma leve diferença de desempenho para $n = 10000$, onde o insertion sort encontra maior dificuldade com arrays aleatórios com repetição. Isto faz sentido para o modo como o algoritmo foi feito: o loop interno somente é interrompido se o item comparado for **menor** que o item a ser inserido. Assim, se houverem vários números iguais ao novo item, o loop interno percorrerá a todos antes de inseri-lo. Nesse caso, a correção é simples: basta estabelecer uma verificação de **menor ou igual** (o que também faz com que o algoritmo passe a ser estável).

É um indicativo de erro de lógica básica no código que se desenvolve em um problema real de complexidade.

D. Heap Sort

No gráfico do heap sort, nota-se uma consistência: todos os pontos tiveram o mesmo comportamento de função. Isso confirma a análise teórica de limite superior e inferior denotados pela mesma complexidade.

Apesar de os limites superior e inferior serem definidos pela mesma função principal, eles conseguem envolver adequadamente os quatro casos de array: aproximam-se dos pontos para n pequeno e distanciam-se para n grande. Esse comportamento atesta a validade das assíntotas para $n \rightarrow \infty$ (por análise gráfica).

É possível ver que as curvas de complexidade para tipos de vetores se agruparam em duplas.

A primeira dupla é a de arrays ordenados e arrays decrescentes. Os dois apresentaram um desempenho muito similar, o que é curioso, dada tamanha distinção entre os dois. Assim, apesar de lidar mal com arrays ordenados em comparação ao insertion sort, o heap sort tem uma organização a tal ponto de tornar a ordenação de arrays decrescentes mais rápida do que a ordenação de arrays aleatórios comuns.

A segunda dupla é a de arrays aleatórios e aleatórios com repetição. Os dois também apresentaram um desempenho muito similar, com os arrays aleatórios sem repetição apresentando uma menor complexidade, assim como no insertion sort.

VI. DADOS EMPÍRICOS (BASELINE)

Os dados a seguir são de algoritmos desconhecidos, fornecidos pelo professor.

TABLE V
TEMPO DE EXECUÇÃO (MS) - ARRAY ORDENADO

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Alg A	0.199	10.47	37.50	1177.1	4997.5
Alg B	0.199	2.294	3.787	15.36	36.48
Alg C	45.61	187.79	392.4	1999.0	3886.4
Alg D	0.0	1.563	3.124	17.277	40.779

TABLE VI
TEMPO DE EXECUÇÃO (MS) - ARRAY DECRESCENTE

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Alg A	0.497	10.47	60.94	1379.2	5752.4
Alg B	0.194	1.599	3.586	22.54	32.71
Alg C	39.89	218.8	397.8	2032.3	4234.9
Alg D	0.0	1.577	3.13	14.105	41.129

TABLE VII
TEMPO DE EXECUÇÃO (MS) - ARRAY ALEATORIZADO

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Alg A	0.0	9.375	57.25	1440.5	5479.5
Alg B	0.199	1.395	2.991	16.45	34.61
Alg C	40.17	194.2	404.0	2071.7	4153.1
Alg D	0.0	1.563	3.16	15.738	37.591

TABLE VIII
TEMPO DE EXECUÇÃO (MS) - ARRAY ALEATORIZADO COM REPETIÇÃO

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Alg A	0.595	13.76	47.07	1242.4	5286.8
Alg B	0.299	1.594	3.972	16.06	32.99
Alg C	43.18	200.5	375.5	2080.7	4132.3
Alg D	0.0	1.560	3.126	15.698	31.393

VII. ANÁLISE EMPÍRICA (BASELINE)

A. Array ordenado

Para o array ordenado, houveram resultados bem variados entre os quatro algoritmos.

O melhor algoritmo para $n \in [100, 1000]$ é Alg D, e o segundo melhor, Alg B. Mas, para $n \geq 5000$, os papéis se invertem: Alg B passa a ser o melhor e Alg D passa a ser o pior. Isso mostra que, para arrays ordenados, Alg D é o melhor se o tamanho for pequeno, e Alg B se destaca em arrays grandes.

O Alg A inicia com uma complexidade muito competitiva para $n = 100$, mas a complexidade piora cada vez mais a partir de $n = 500$. O mesmo ocorre para Alg C, que inicia com tempos de execução muito maiores que Alg A, mas começa a ganhar de Alg A em $n = 10000$. Disto, conclui-se que Alg A é melhor que Alg C para arrays pequenos, mas Alg C é melhor que Alg A para arrays grandes. Porém, ainda assim, possuem uma eficiência muito ruim comparados a Alg B e Alg D.

Para este caso em específico, Alg A e Alg C aparentam ter complexidade muito maior que Alg B e Alg D, que, possuem melhor desempenho para qualquer n , exceto em casos específicos para n pequeno.

B. Array decrescente

Para o array decrescente, os comportamentos continuam similares, com exceção de que o Alg B necessita de uma quantidade maior de entradas para superar o Alg D, o que mostra uma dificuldade do Alg B lidar com arrays em ordem inversa.

C. Array Aleatorizado

No array aleatorizado, fica mais evidente que Alg C é uma melhor opção para n grande comparado a Alg A, enquanto Alg A é bem mais eficiente para n pequeno.

É válido notar que Alg A atingiu a marca de 0.0 ms para $n = 100$, junto com Alg D, o que mostra que os dois lidam muito bem com arrays pequenos nessa situação.

Alg B e Alg D ficam próximos em desempenho, alternando entre si na primeira e segunda posição de algoritmos mais eficientes. É difícil dizer qual dos dois é melhor para n pequeno e grande.

D. Array Aleatorizado com Repetição

Alg A e Alg C mantiveram o mesmo comportamento do Array Aleatorizado. Alg D mostra ter um desempenho melhor que Alg B para todo n .

E. Conclusão

Os algoritmos A e C são descartáveis, uma vez que possuem desempenho ruim em todos os casos, exceto em casos específicos, como em $n = 100$ para Alg A em arrays aleatorizados, mas tal marco também é alcançado por Alg D. Os algoritmos A e C possuem uma complexidade muito maior do que os algoritmos B e D (podendo inclusive ser quadrática), fato visível quando $n = 1000$ e o custo de Alg B e Alg D é cerca de 100 vezes menor.

Já quanto aos algoritmos B e D, a escolha do algoritmo depende do caso. Tanto no array crescente quanto no decrescente Alg B possui melhor desempenho para $n \rightarrow \infty$, enquanto Alg D possui melhor desempenho para n pequeno. Nos arrays aleatorizados e aleatorizados com repetição, o desempenho dos dois é similar, mas Alg B sempre começa ligeiramente pior em $n = 100$ do que Alg D.

Esse comportamento de começar mal e melhorar depois é característico de assíntotas que se interceptam, como $\mathcal{O}(n)$ com coeficiente grande e $\mathcal{O}(n^2)$ com coeficiente pequeno. Isso mostra que, se Alg B possui melhor desempenho para $n \rightarrow \infty$, então a complexidade de Alg B é menor que a Alg D. A escolha entre Alg B e Alg D dependerá do uso, com ambos possuindo vantagens e desvantagens.

VIII. DIFICULDADES ENFRENTADAS

Não houveram dificuldades significativas no trabalho.

REFERENCES

- [1] P. Feofiloff, "A estrutura heap." Accessed: Oct. 03, 2026. [Online]. Available: https://www.ime.usp.br/~pf/analise_de_algoritmos/aulas/heap.html
- [2] "Fórmula de Stirling." Accessed: Oct. 03, 2026. [Online]. Available: https://pt.wikipedia.org/wiki/F%C3%B3rmula_de_Stirling

IX. APÊNDICE A: TEMPO DE EXECUÇÃO (MS) DOS ALGORITMOS ESCOLHIDOS

TABLE IX
TEMPO DE EXECUÇÃO (MS) - ARRAY ORDENADO

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Insertion Sort	0.001	0.003	0.004	0.022	0.045
Heap Sort	0.014	0.100	0.143	0.858	1.833

TABLE X
TEMPO DE EXECUÇÃO (MS) - ARRAY DECRESCENTE

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Insertion Sort	0.030	0.718	1.890	48.34	198.61
Heap Sort	0.011	0.088	0.133	0.821	1.804

TABLE XI
TEMPO DE EXECUÇÃO (MS) - ARRAY ALEATÓRIO

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Insertion Sort	0.018	0.280	0.968	24.13	98.03
Heap Sort	0.013	0.075	0.153	0.989	2.187

TABLE XII
TEMPO DE EXECUÇÃO (MS) - ARRAY ALEATÓRIO COM REPETIÇÃO

Algoritmo	n=100	n=500	n=1000	n=5000	n=10000
Insertion Sort	0.016	0.224	0.989	23.93	105.60
Heap Sort	0.013	0.066	0.154	0.987	2.250